

```

"""
Weekly Eyewear & Eye Health Trends Dashboard
- Fetches Google Trends data via SerpAPI
- Logs historical data to Google Sheets
- Sends a formatted HTML email every Monday
"""

import requests
import smtplib
import json
import os
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from datetime import datetime
import gspread
from google.oauth2.service_account import Credentials

# -----
# CONFIG - all sensitive values are read from environment variables
# (set these as GitHub Actions secrets)
# -----
SERPAPI_KEY = os.environ["SERPAPI_KEY"]
EMAIL_FROM = os.environ["EMAIL_FROM"]
EMAIL_TO = os.environ["EMAIL_TO"]
EMAIL_PASSWORD = os.environ["EMAIL_PASSWORD"] # Gmail App Password
GSHEET_ID = os.environ["GSHEET_ID"] # Google Sheet ID from URL
GOOGLE_CREDS_JSON = os.environ["GOOGLE_CREDS_JSON"] # Full service-account JSON

# -----
# KEYWORDS - edit these to match your tracking needs
# -----
KEYWORDS = [
    # Eye health
    "eye exam", "dry eyes", "eye drops", "vision correction",
    # Eyewear styles
    "blue light glasses", "prescription sunglasses", "contact lenses",
    # Trend terms
    "oversized glasses", "cat eye glasses", "transition lenses",
]

SERPAPI_URL = "https://serpapi.com/search"

# -----

```

```

# 1. FETCH TRENDS
# -----
def fetch_trends() -> dict:
    """Return avg interest, peak, and direction for each keyword (last 7 days)."""
    results = {}
    chunks = [KEYWORDS[i:i+5] for i in range(0, len(KEYWORDS), 5)]

    for chunk in chunks:
        params = {
            "engine": "google_trends",
            "q":      ", ".join(chunk),
            "date":   "now 7-d",
            "geo":    "US",
            "api_key": SERPAPI_KEY,
        }

        resp = requests.get(SERPAPI_URL, params=params, timeout=30)
        resp.raise_for_status()
        data = resp.json()

        timeline = data.get("interest_over_time", {}).get("timeline_data", [])

        for kw in chunk:
            values = []
            for point in timeline:
                for val in point.get("values", []):
                    if val["query"] == kw:
                        values.append(val.get("extracted_value", 0))

            if values:
                results[kw] = {
                    "avg": round(sum(values) / len(values), 1),
                    "peak": max(values),
                    "trend": "📈" if values[-1] > values[0] else "📉",
                    "values": values,  # kept for Sheets logging
                }

    return results

# -----
# 2. FETCH RISING QUERIES (top 3 for a representative keyword)
# -----
def fetch_rising(keyword: str) -> list[str]:
    params = {
        "engine": "google_trends",
        "q":      keyword,
        "date":   "now 7-d",
    }

```

```

        "data_type": "RELATED_QUERIES",
        "geo":        "US",
        "api_key":    SERPAPI_KEY,
    }
    resp = requests.get(SERPAPI_URL, params=params, timeout=30)
    data = resp.json()
    rising = data.get("related_queries", {}).get("rising", [])
    return [r["query"] for r in rising[:3]]

# -----
# 3. LOG TO GOOGLE SHEETS
# -----
def log_to_sheets(data: dict, week_label: str):
    """
    Appends one row per keyword to a 'Weekly Log' sheet.
    Columns: Week | Keyword | Avg Interest | Peak | Direction
    Creates the sheet + header row automatically on first run.
    """
    creds_info = json.loads(GOOGLE_CREDS_JSON)
    scopes = [
        "https://www.googleapis.com/auth/spreadsheets",
        "https://www.googleapis.com/auth/drive",
    ]
    creds = Credentials.from_service_account_info(creds_info, scopes=scopes)
    client = gspread.authorize(creds)

    spreadsheet = client.open_by_key(GSHEET_ID)

    # Get or create the log sheet
    try:
        sheet = spreadsheet.worksheet("Weekly Log")
    except gspread.WorksheetNotFound:
        sheet = spreadsheet.add_worksheet(title="Weekly Log", rows=1000, cols=10)
        sheet.append_row(["Week", "Keyword", "Avg Interest (0-100)", "Peak", "Dir

rows = []
for kw, stats in data.items():
    rows.append([week_label, kw, stats["avg"], stats["peak"], stats["trend"]])

sheet.append_rows(rows)
print(f"✅ Logged {len(rows)} rows to Google Sheets for week {week_label}")

# -----
# 4. BUILD HTML EMAIL
# -----

```

```

def build_html(data: dict, rising: list[str], week_label: str) -> str:
    # Sort by avg interest descending
    sorted_kws = sorted(data.items(), key=lambda x: -x[1]["avg"])

    rows = ""
    for kw, stats in sorted_kws:
        bg = "#fff"
        rows += f"""
        <tr style="background:{bg};">
            <td style="padding:8px 12px;">{kw}</td>
            <td style="padding:8px 12px; text-align:center;">{stats['avg']}</td>
            <td style="padding:8px 12px; text-align:center;">{stats['peak']}</td>
            <td style="padding:8px 12px; text-align:center;">{stats['trend']}</td>
        </tr>"""

    rising_html = ""
    if rising:
        rising_items = "".join(f"<li>{q}</li>" for q in rising)
        rising_html = f"""
        <h3 style="color:#444;">🔥 Rising Searches This Week</h3>
        <ul style="font-size:14px; color:#333;">{rising_items}</ul>"""

    return f"""
    <html>
    <body style="font-family: Arial, sans-serif; background:#f9f9f9; padding:24px"
    <div style="max-width:620px; margin:auto; background:#fff; border-radius:8px
        box-shadow:0 2px 8px rgba(0,0,0,0.08); padding:32px;">

        <h2 style="color:#1a1a2e; margin-top:0;">🕶️ Weekly Eyewear & Eye Health T
        <p style="color:#666; font-size:14px;">Week of <strong>{week_label}</stro
        Source: Google Trends via SerpAPI &nbsp;|&nbsp; Scale: 0-100</p>

        <table style="width:100%; border-collapse:collapse; font-size:14px;">
            <thead>
                <tr style="background:#1a1a2e; color:#fff;">
                    <th style="padding:10px 12px; text-align:left;">Keyword</th>
                    <th style="padding:10px 12px;">Avg Interest</th>
                    <th style="padding:10px 12px;">Peak</th>
                    <th style="padding:10px 12px;">Direction</th>
                </tr>
            </thead>
            <tbody>{rows}</tbody>
        </table>

        {rising_html}

        <p style="font-size:12px; color:#aaa; margin-top:24px; border-top:1px sol

```

```

        This report is auto-generated every Monday. Historical data is logged i
    </p>
</div>
</body>
</html>""""

```

```

# -----
# 5. SEND EMAIL
# -----

def send_email(html: str, week_label: str):
    msg = MIMEMultipart("alternative")
    msg["Subject"] = f"🕶 Eye Trends Report - {week_label}"
    msg["From"] = EMAIL_FROM
    msg["To"] = EMAIL_TO
    msg.attach(MIMEText(html, "html"))

    with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
        server.login(EMAIL_FROM, EMAIL_PASSWORD)
        server.sendmail(EMAIL_FROM, EMAIL_TO, msg.as_string())
    print(f"✅ Email sent to {EMAIL_TO}")

# -----
# MAIN
# -----

if __name__ == "__main__":
    week_label = datetime.now().strftime("%B %d, %Y")
    print(f"▶ Running trends dashboard for {week_label} ...")

    print(" Fetching trends from SerpAPI ...")
    data = fetch_trends()

    print(" Fetching rising queries ...")
    rising = fetch_rising("blue light glasses") # pick a representative keyword

    print(" Logging to Google Sheets ...")
    log_to_sheets(data, week_label)

    print(" Building and sending email ...")
    html = build_html(data, rising, week_label)
    send_email(html, week_label)

    print(f"✅ Done.")

```